

UNIVERSITY Of Brahmanbaria

Department of Computer Science & Engineering

System Design & Architecture Document

DSA Trivia Challenge

A Console-Based Competitive Quiz & Ranking System in C

Course Name	Data Structure & Algorithm Design
Course Code	CSE 0613-2107
Document Type	Design Diagram
Prepared By	Group 05
Supervised By	Tanzila Akter Pushpu
Submission Date	10 October 2026
Version	8.6 (Last Update 07 October 2026)

GROUP MEMBERS

SL	Student Name	Registration No.	Batch	Role
01	Ashraful Islam	25201000	252-CSE	Team Leader / Developer
02			252-CSE	Documentation
03			252-CSE	Research & Slides
04			252-CSE	Testing & Q&A

1. Executive Summary

The **DSA Trivia Challenge (Version 2.0)** is a console-based multiple-choice quiz application for university students. A candidate registers with a Student/Roll ID, answers 10 randomly selected questions from a 60-question bank under a ``+1.00 / -0.25 / 0.00`` marking scheme, and is ranked on an accuracy-weighted **Fair Ranking Score** using competition ranking (``1, 2, 2, 4 ...``).

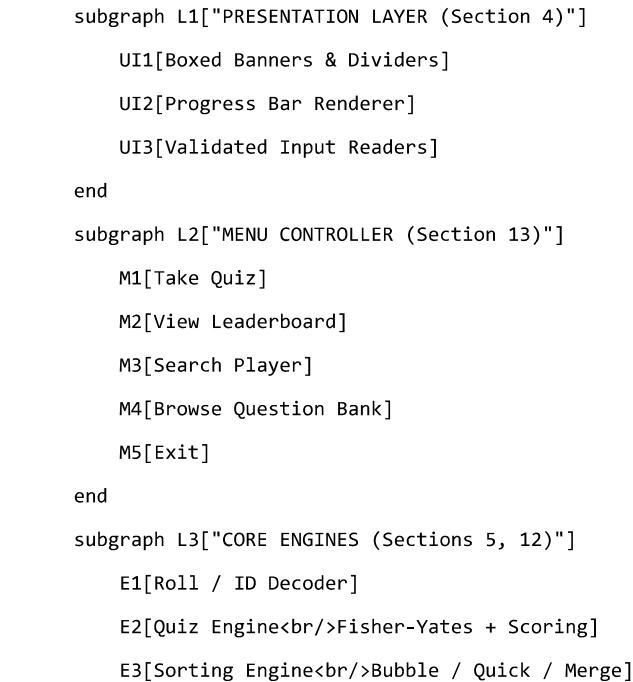
The program is deliberately built **from scratch in pure C** to demonstrate ten classic data structures and algorithms working together inside one coherent system:

#	DSA Concept	Where It Lives in the Code
1	Singly Linked List	Dynamic question bank (60 MCQs) — <code>`QNode`</code>
2	Hash Table	$O(1)$ average player authentication — <code>`HashTable`</code>
3	Binary Search Tree	Player records indexed by Student ID — <code>`Player`</code> tree
4	Bubble Sort	$O(n^2)$ leaderboard baseline — <code>`bubble_sort_players()`</code>
5	Quick Sort	$O(n \log n)$ average leaderboard — <code>`quick_sort_players()`</code>
6	Merge Sort	$O(n \log n)$ stable leaderboard — <code>`merge_sort_players()`</code>
7	Fisher–Yates Shuffle	Fair random question selection — <code>`shuffle_indices()`</code>
8	Roll/ID Decoder	Parses 7–8 digit IDs → Year / Batch / Serial
9	Fair Ranking Score	Accuracy-weighted composite metric — <code>`finalize_scores()`</code>
10	Competition Ranking	Tied scores share a rank — <code>`assign_competition_ranks()`</code>

2. High-Level System Architecture

The program follows a **layered, menu-driven architecture**. Each layer only talks to the layer below it, keeping the ten DSA modules independent and testable.

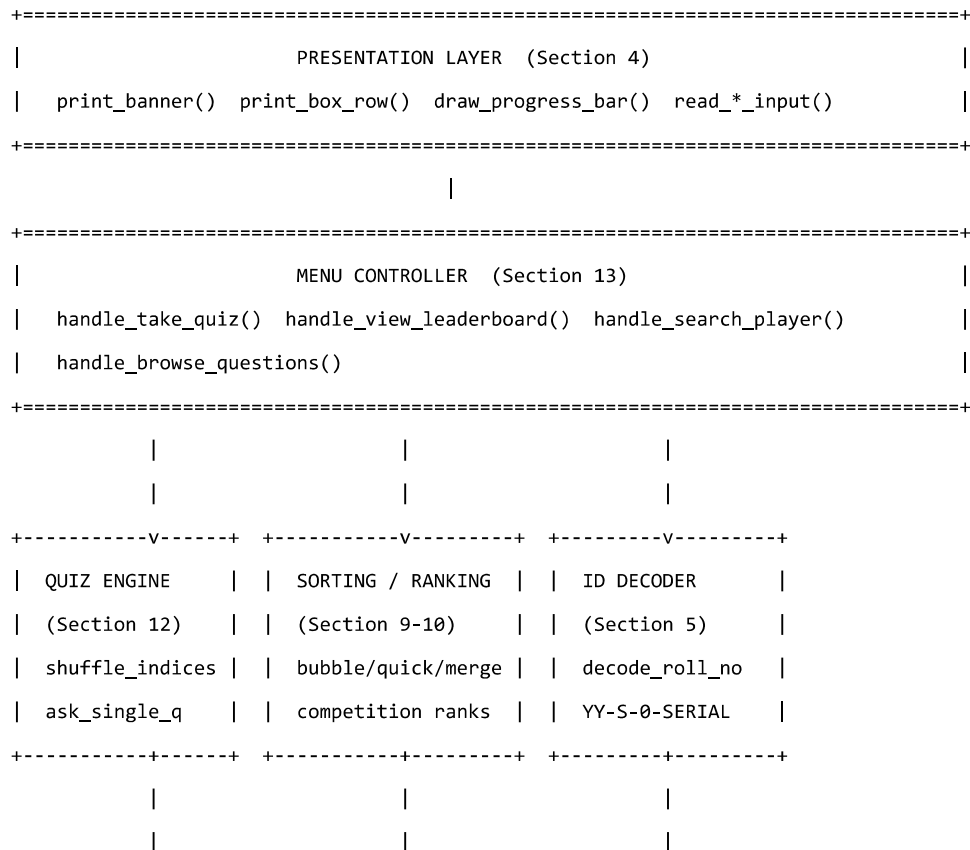
flowchart TB

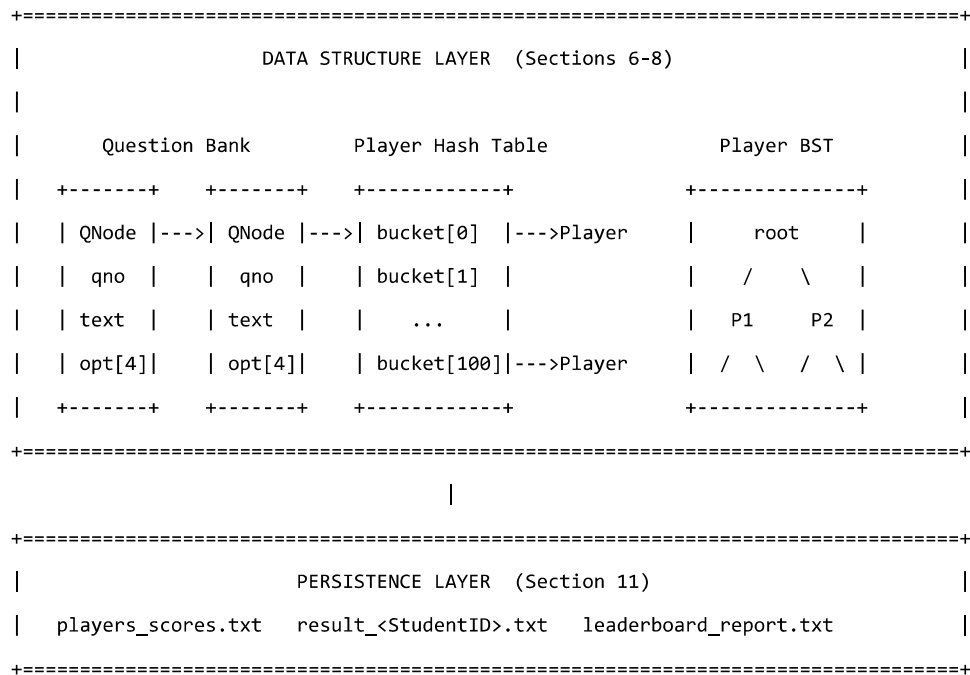


System Design & Architecture Document

```
    E4[Ranking Engine<br/>Competition Ranks]
end
subgraph L4["DATA STRUCTURE LAYER (Sections 6-8)"]
    D1[Singly Linked List<br/>60 Question Nodes]
    D2[Hash Table ×101<br/>DJB2 + Chaining]
    D3[Binary Search Tree<br/>Player Records by ID]
end
subgraph L5["PERSISTENCE LAYER (Section 11)"]
    F1[(players_scores.txt)]
    F2[(result_<ID>.txt)]
    F3[(leaderboard_report.txt)]
end
L1 --> L2 --> L3 --> L4 --> L5
```

2.1 ASCII View

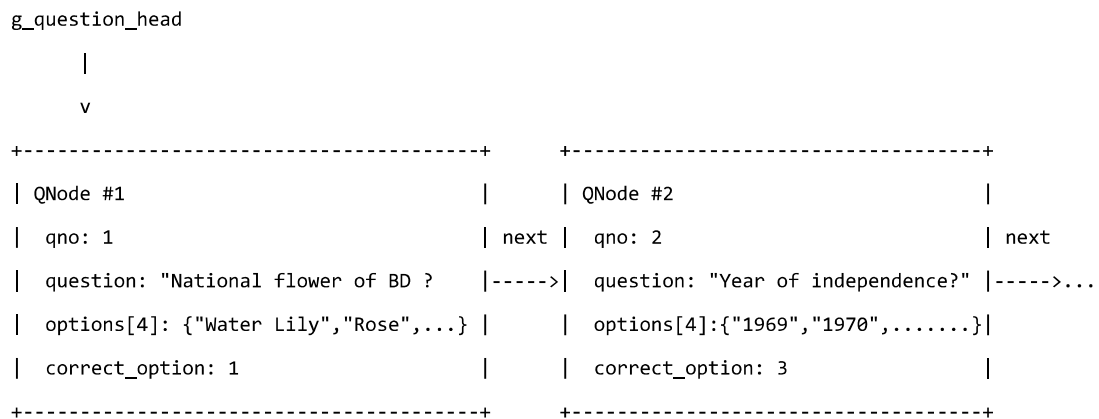




3. Data Structures — Detailed Design

3.1 Singly Linked List — Question Bank

The entire 60-question bank is a **singly linked list of `QNode` structs**, built at startup by `init\_question\_bank()` and grown with tail-insertion in `add\_question()`. Using a linked list (not an array) demonstrates dynamic allocation; traversal is $O(n)$, insertion is $O(1)$ at the tail ($O(n)$ to find the tail).



Node definition

```

typedef struct QNode {
    int qno;                /* question number (1..40, 101..120) */
    char question[MAX_Q_TEXT]; /* 256 chars */
    char options[4][MAX_OPT_LEN]; /* 4 x 128 chars */

```

System Design & Architecture Document

```
int correct_option;          /* 1..4 (human friendly, 1-based) */
struct QNode* next;
} QNode;
```

Note: question IDs are **non-contiguous** (`1–40` and `101–120`). All lookups walk the list and compare `qno`, which keeps the code uniform.

3.2 Hash Table — Player Authentication (DJB2 + Separate Chaining)

Player authentication must be fast: entering an ID triggers a lookup during registration, retake checks, profile search, and question-bank access control. The program hashes the ID string with the **DJB2** algorithm (`hash = hash\*33 + tolower(c)`, seed 5381) into **101 buckets** with **separate chaining**.

```
hash_function("2520853") = h
                        |
                        v
+-----+
| HASH TABLE (HASH_SIZE = 101) | +-----+
| buckets[0] -----NULL | |Player |
| buckets[1] ---+         | +-----+
|   ...         |         |
| buckets[h] ---+--> Player---|---> Player --NULL (chain)
|   ...         | (id "252...") | (collision)
| buckets[100] -----NULL |
+-----+
```

Key design decision — shared allocation: the `Player` node inserted into the hash chain is **the same heap object** that lives in the BST. `Player.hash\_next` lets one node serve both structures — **no duplicate allocation, no synchronisation problem**.

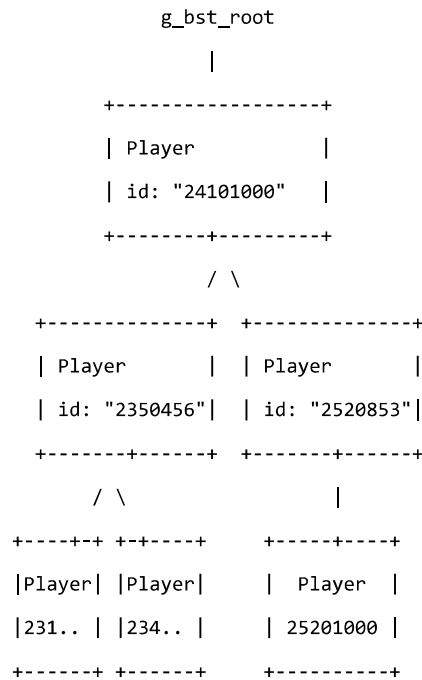
```
typedef struct {
    Player* buckets[HASH_SIZE];
} HashTable;

unsigned long hash_function(const char* str) {
    unsigned long hash = 5381;
    int c;
    while ((c = (unsigned char)*str++))
        hash = ((hash << 5) + hash) + (unsigned long)tolower(c);
    return hash % HASH_SIZE;
}
```

Operation	Complexity
`hash_insert`	O(1) average
`hash_lookup` (authentication)	O(1) average, O(n) worst case
Case-insensitivity	`tolower()` inside the hash — `"2520853" ≡ "2520853 "` variants of case

3.3 Binary Search Tree — Player Records Indexed by ID

All player records form a **BST ordered lexicographically by Student ID**. The BST provides in-order traversal for file saving (``bst_inorder_collect``) and sorted-ID iteration, while the hash table provides fast point lookups. Together they are a classic **"BST + Hash" hybrid index**.



Every Player box above ALSO sits in one hash bucket via `hash_next`.

Operation	Complexity
<code>`bst_insert`</code>	$O(\log n)$ average / $O(n)$ degenerate
<code>`bst_search`</code>	$O(\log n)$ average
<code>`bst_inorder_collect`</code>	$O(n)$ — yields players sorted by ID
<code>`bst_free`</code>	Post-order $O(n)$ — no leaks on exit

3.4 The `Player` Record — One Node, Two Structures

```

typedef struct Player {
    char name[MAX_NAME_LEN];    /* identity block          */
    char id[MAX_ID_LEN];
    char batch[MAX_BATCH_LEN];  /* first 3 digits of ID    */
    char dept[MAX_DEPT_LEN];
    int total_questions;        /* stats block             */
    int attempted, correct, wrong, skipped;
    double raw_score;           /* >= 0 (+1.00 / -0.25)   */
    double accuracy;           /* correct/attempted * 100 */
}
  
```

```
double ranking_score;      /* accuracy-weighted      */
int    rank;               /* competition rank      */
struct Player* left;       /* BST links              */
struct Player* right;
struct Player* hash_next;  /* hash chain link       */

} Player;
```

Retake-update safety: when an existing player retakes the quiz, the code copies the new stats into the **existing** heap node and **preserves** ``left``, ``right``, and ``hash_next`` before overwriting — otherwise the BST and hash chains would be corrupted by the struct assignment `*existing = p``.

3.5 Per-Answer Log — ``AnswerRecord``

A plain array on the stack records ``questionId`` → `selectedOption`` (0 = skipped) so the detailed transcript file can be written **after** the quiz without re-asking anything.

4. Algorithms — Detailed Design

4.1 Fisher–Yates Shuffle — Fair Question Selection

```
void shuffle_indices(int arr[], int n) {
    for (int i = n - 1; i > 0; i--) {
        int j = rand() % (i + 1);          /* unbiased: j in [0, i] */
        int tmp = arr[i]; arr[i] = arr[j]; arr[j] = tmp;
    }
}
```

First ``count_questions()`` nodes are collected into a temporary array, their indices are shuffled, and the **first 10 indices** pick the session questions. Every question is equally likely — no repetition within an attempt.

Entropy hardening: a static counter is mixed with ``time(NULL)`` and ``clock()`` before every session (``srand(time ^ clock ^ counter)``) so rapid repeated attempts in the same second don't reproduce the same order.

4.2 Scoring — Raw Score, Accuracy, Fair Ranking Score

```
+-----+
correct answer | raw_score += 1.00      |
wrong answer  | raw_score -= 0.25 (min 0.00) |
skip          | no change          |
+-----+

|
accuracy = correct / attempted x 100
|
```

```

+-----+
| IF raw_score > 0:                |
|     ranking_score = raw_score x (0.50 + 0.50 x acc/100) |
| ELSE:                            |
|     ranking_score = raw_score    |
+-----+

```

Why the accuracy weighting? Volume alone is not skill. A player who attempts all 10 and gets 6 right (raw = 5.5) is rewarded more than one who guesses wildly: the factor ranges from 0.50 (0% accuracy) to 1.00 (100% accuracy). Skips never reduce the score, but unanswered questions drag accuracy down indirectly (denominator shrinks).

Player	Correct / Wrong / Skip	Raw	Accuracy	Factor	Ranking Score
A	10 / 0 / 0	10.00	100%	1.00	10.00
B	8 / 2 / 0	7.50	80%	0.90	6.75
C	6 / 0 / 4	6.00	100%	1.00	6.00
D	9 / 8 / 0	7.00	53%	0.765	5.36
E	0 / 4 / 6	0.00	0%	—	0.00

4.3 Sorting Engine — One Comparator, Three Algorithms

All three sorts call the **same comparator**, so the leaderboard order is algorithm-independent; only the running time differs. This makes the menu's "choose your sort" option a genuine live demo of complexity theory.

Comparator priority (descending):

raw_score → accuracy → raw_score → id (ascending, deterministic tiebreak)

```
int compare_players(const Player* a, const Player* b);
```

Algorithm	Best	Average	Worst	Stable	Space
Bubble (bubble_sort_players)	O(n)	O(n ²)	O(n ²)	Yes	O(1)
Quick (Lomuto partition_players)	O(n log n)	O(n log n)	O(n ²)	No	O(log n) stack
Merge (merge_players)	O(n log n)	O(n log n)	O(n log n)	Yes	O(n) temp

Each run is timed with clock() and the elapsed milliseconds are shown on screen and written into leaderboard_report.txt.

4.4 Competition Ranking

After sorting, assign_competition_ranks() walks the array: players whose raw_score differs by less than EPS = 1e-6 **share the previous rank**; otherwise the rank jumps to position + 1.

Sorted scores: 10.00 6.75 6.75 6.00 0.00 0.00

Ranks: 1 2 2 4 5 5 -> (1, 2, 2, 4, 5, 5)

4.5 Roll / Student ID Decoder

ID layout (7 or 8 digits, all numeric, 4th digit = '0'):

"2 5 2 0 8 5 3"

| | | | |----- SERIAL = "853" (3 or 4 digits)

| | | | +----- divider = '0' (fixed)

| | +---+----- semester= 2 (1 = Spring, 2 = Fall)

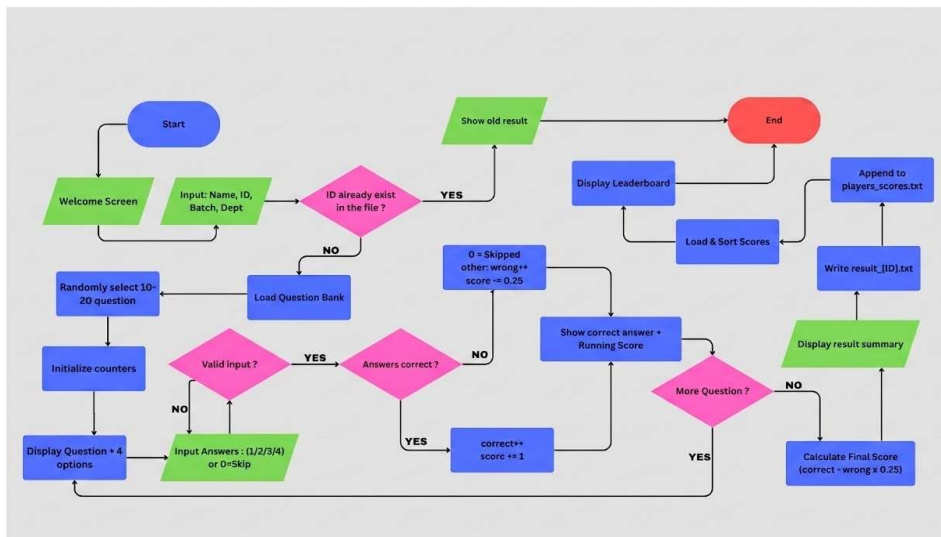
+---+----- year = "25" -> 2025

First three digits = Batch: "252"

Output: "Admission Year: 2025 | Batch: 252 | Serial: 853"

5. Program Flowcharts

5.1 Main Loop



5.2 Leaderboard View (`handle\_view\_leaderboard`)

```
+-----+ +-----+
| Collect BST nodes | --> | Prompt: Bubble/Quick/Merge|
| into pointer arr | +-----+
+-----+
|
+-----v-----+
| clock() start |
| run chosen sort |
| clock() end |
+-----+-----+
|
```

```

+-----v-----+
| assign_competition_ranks() |
+-----+-----+
|
+-----+-----+
| Print top-15 table with timing info |
| export_leaderboard_report(file)    |
+-----+-----+

```

6. Module / Function Reference

Section	Module	Key Functions
4	UI Utilities	`print_banner`, `print_box_row`, `print_fixed_row`, `draw_progress_bar`, `read_string_input`, `read_integer_input`, `pause_prompt`
5	ID Decoder	`decode_roll_no`
6	Question Bank (LL)	`add_question`, `init_question_bank`, `count_questions`, `display_question_bank`, `free_question_bank`
7	Hash Table	`init_hash_table`, `hash_function` (DJB2), `hash_insert`, `hash_lookup`
8	BST	`bst_insert`, `bst_search`, `bst_count`, `bst_inorder_collect`, `bst_free`
9	Sorting	`compare_players`, `bubble_sort_players`, `partition_players`, `quick_sort_players`, `merge_players`, `merge_sort_players`, `assign_competition_ranks`
10	Scoring	`apply_answer_result`, `finalize_scores`
11	File I/O	`save_all_players`, `load_all_players`, `export_leaderboard_report`, `write_result_file`
12	Quiz Engine	`shuffle_indices`, `ask_single_question`, `run_quiz_session`
13	Menu Handlers	`show_main_menu`, `handle_take_quiz`, `handle_view_leaderboard`, `handle_search_player`, `handle_browse_questions`, `display_player_card`
14	Main	`main`

7. File Formats (Runtime Artifacts)

7.1 `players\_scores.txt` — pipe-delimited, one record per line, rewritten on every save

```
name|id|batch|dept|total_q|attempted|correct|wrong|raw_score|accuracy|ranking_score
```

```
Rahim Uddin|2520853|252|CSE|10|10|8|2|7.50|80.00|6.75
```

Loaded with a single `scanf` of 11 fields; malformed lines are skipped (`matched != 11`). `skipped` is re-derived as `total\_q - attempted`. Ranks are **not** persisted — they are recomputed at startup by a merge sort.

7.2 `result\_<StudentID>.txt` — per-attempt transcript

Header (identity + decoded roll info + timestamp) → question-by-question breakdown ([CORRECT]/[WRONG]/[SKIPPED]), the chosen option text, and the correct answer when wrong) → final score block → rank.

7.3 `leaderboard\_report.txt` — snapshot table

Fixed-width table with the algorithm name, its measured time in milliseconds, competition-rank legend, and all ranked rows.

8. Complexity Summary

Feature	Structure / Algorithm	Time	Space
Question storage	Singly linked list	access $O(n)$, insert $O(1)^*$	$O(1)$ per node
Player authentication	Hash table	$O(1)$ avg lookup	$O(n)$ total
Player enumeration by ID	BST in-order	$O(n)$	$O(n)$
Question randomisation	Fisher–Yates	$O(n)$	$O(n)$ temp
Leaderboard (baseline)	Bubble sort	$O(n^2)$	$O(1)$
Leaderboard (fast)	Quick / Merge sort	$O(n \log n)$	$O(\log n) / O(n)$
Rank assignment	Single pass	$O(n)$	$O(1)$

9. Build & Run

```
gcc -Wall -Wextra -std=c11 DSA_TRIVIA.c -o dsa_trivia./dsa_trivia
```

- ANSI colour codes degrade gracefully on dumb terminals.
- `CLEAR\_SCREEN` switches between `cls` (Windows) and `clear` (POSIX) at compile time.
- Input is read with `fgets` everywhere — no `scanf` buffer-overrun hazards; the integer reader additionally rejects `5x`-style trailing garbage.

10. Design Highlights & Defensive Decisions

1. **One node, two indexes** — `Player` serves BST and hash table simultaneously; retakes surgically preserve link fields across the overwrite.
2. **Single comparator** guarantees identical leaderboard semantics across all three sorting algorithms; only wall-clock time differs.
3. **Accuracy-weighted ranking** resists brute-force guessing, a classic fairness exploit in negative-marking exams.
4. **Transcript integrity** — `AnswerRecord` logs make the per-question report reproducible after the fact.
5. **Persistence is append-free** — the full database is rewritten atomically per save, so the file never contains half-written records.
6. **Graceful degradation** — free-form IDs, terminals without ANSI support, and malformed save-file lines are all handled without crashing.